FACULDADE SÃO PAULO TECH SCHOOL
CIÊNCIA DA COMPUTAÇÃO

StreamGuard

Monitoramento de Componentes Físicos de Sites de Streaming

Anderson Francisco Santos Oliveira
Caio Picciarelli Silva
Kauê Tamura Gomes
Leonardo Werner Silva
Marina Santos Paixão Ribeiro

São Paulo
2026

Documentação técnica — versão revisada e ampliada

Sumário

Sumário 2
1. Contexto 4
2. Problema 4
3. Objetivo 4
3.1 Objetivos específicos 5
4. Justificativa 5
5. Escopo 5
6. Premissas e Restrições 6
Premissas 6
Restrições 6
7. Requisitos do Sistema 6
7.1 Requisitos Funcionais 6
7.2 Requisitos Não Funcionais 7
8. Arquitetura da Solução 7
8.1 Camada de coleta 8
8.2 Camada de backend e persistência 8
8.3 Motor de alertas 8
8.4 Camada de notificação e rastreabilidade 9
8.5 Camada de apresentação 9
9. Modelo de Dados 9
9.1 Entidades principais 9
10. Regras de Negócio 10
10.1 Contas, empresas e permissões 10
10.2 Equipamentos, componentes e métricas 10
10.3 Alertas e notificações 10
11. Integração com Ferramentas Externas (Slack e Jira) 11
11.1 Por que Incoming Webhooks 11
11.2 Estrutura da mensagem (payload JSON) 11
11.3 Jira — abertura automática de chamado 11
11.4 Limitações conhecidas e possibilidades de evolução 11
12. Fluxo de Alertas e Notificações 12
13. Interfaces do Sistema — Wireframes 12
14. User Stories 17
14.1 Persona: Gestor de Infraestrutura (Maicon) 18
14.2 Persona: Analista de Dados (Kleber) 18
15. Metodologia 24
Comunicação 24
Transparência e Visibilidade 24

Melhoria Contínua 24
16. Diagrama de Requisitos — Backlog 24
17. Gráfico de Burndown 26
18. Considerações Finais 27
19. Referências Bibliográficas 28
Contexto e justificativa 28
Fundamentação técnica e normativa 28

1. Contexto

Durante a Copa do Mundo de 2026, a transmissão da CazéTV no YouTube atingiu recordes mundiais de audiência, chegando a mais de 23 milhões de pessoas assistindo simultaneamente à final — um aumento de 114% em relação à última Copa, em 2022.

O sucesso, no entanto, expôs uma fragilidade: muitos usuários relataram travamentos e congelamento de imagem justamente nos momentos mais importantes dos jogos, mesmo possuindo boa conexão de internet em casa. Isso evidencia que o problema pode estar relacionado à infraestrutura responsável pela transmissão, e não necessariamente à conexão do usuário.

Quando milhões de pessoas acessam o mesmo conteúdo simultaneamente, os servidores e demais componentes da infraestrutura podem ficar sobrecarregados, apresentando alto uso de processamento, memória, armazenamento ou capacidade de rede. Esse cenário pode provocar falhas perceptíveis ao público, como travamentos, queda na qualidade da imagem e até interrupções completas da transmissão.

Esse tipo de pico de acesso tende a se repetir em outros grandes eventos esportivos, como as próximas Olimpíadas, tornando o monitoramento da infraestrutura uma necessidade cada vez mais relevante.

2. Problema

Atualmente, não existe um mecanismo automatizado e centralizado para monitorar, em tempo real, o estado dos componentes físicos e lógicos — CPU, memória, armazenamento, rede e serviços — que sustentam uma transmissão ao vivo.

Como consequência, sinais de sobrecarga na infraestrutura costumam ser identificados somente depois que já afetam o público, por meio de travamentos, queda de qualidade ou interrupções, e não antes, quando ainda seria possível uma atuação preventiva por parte da equipe técnica.

Esse cenário se agrava em momentos de pico de audiência, como grandes eventos esportivos, quando múltiplos componentes da infraestrutura podem estar sob estresse simultaneamente, dificultando a priorização das ações da equipe técnica sem uma visão consolidada de toda a infraestrutura.

3. Objetivo

Desenvolver um painel dinâmico e automatizado, capaz de coletar em tempo real dados relacionados ao uso dos servidores durante transmissões ao vivo, como:

- Uso de CPU;
- Uso de memória RAM;
- Uso de escrita e leitura de disco;
- Uso e capacidade de rede;
- Status dos principais serviços e componentes.

As informações serão apresentadas em um painel centralizado, permitindo que a equipe responsável acompanhe a infraestrutura em tempo real e emitindo alertas automáticos sempre que os recursos estiverem próximos de seus limites.

3.1 Objetivos específicos

- Permitir que uma mesma conta monitore múltiplas empresas clientes, cada uma com sua própria infraestrutura cadastrada;
- Diferenciar níveis de acesso por perfil de usuário (Administrador, Gerente e Usuário comum), garantindo que cada perfil só execute ações compatíveis com sua responsabilidade;
- Classificar o estado de cada componente monitorado em três níveis de severidade (OK, Alerta e Crítico), com destaque visual por cor;
- Automatizar a comunicação de incidentes por meio de notificações no Slack e abertura de chamados no Jira;
- Manter um histórico rastreável de métricas, alertas e logs para análise posterior.

4. Justificativa

Segundo a WARC Media, eventos esportivos globais movimentaram cerca de US$ 61 bilhões em publicidade em 2024. Quando uma transmissão apresenta problemas durante um momento de grande audiência, cada segundo de buffering pode representar uma oportunidade de publicidade perdida e uma redução da audiência justamente quando ela possui maior valor para anunciantes e patrocinadores.

O impacto dos problemas de buffering também já foi estudado pelo setor. Um estudo da Bitmovin, fornecedora de infraestrutura de streaming, estima que esses problemas podem representar custos de até US$ 390 milhões por mês para a indústria globalmente. O estudo também aponta que 47% dos usuários já cancelaram uma assinatura devido a problemas de buffering; além disso, a tolerância média dos usuários seria de aproximadamente 13,5 segundos de travamento antes de considerarem abandonar a plataforma.

Em relação à infraestrutura de TI de forma geral, dados apresentados pela Splunk e Cisco em 2026 apontam um custo médio de aproximadamente US$ 15 mil por minuto de indisponibilidade em grandes empresas.

Assim, é possível relacionar três fatores importantes: a alta valorização da audiência pelos anunciantes, a baixa tolerância dos usuários a problemas de transmissão e o elevado custo de indisponibilidade da infraestrutura — o que justifica o desenvolvimento de um sistema capaz de antecipar falhas antes que impactem a receita, a audiência e a retenção de usuários.

5. Escopo

O projeto StreamGuard consistirá no desenvolvimento de uma aplicação client e web para monitoramento de componentes de sistemas operacionais (CPU, memória RAM, disco e rede), aplicando os conceitos do ITIL de Monitoramento de Serviços, Gestão de Incidentes e Gestão de Problemas.

- Desenvolvimento das telas Web de cadastro, incluindo o CRUD de seleção dos componentes a serem monitorados;
- Captura de dados de uso de CPU, RAM, disco e rede por meio de um script em Python;
- Painel (dashboard) com gráficos e indicadores para acompanhamento em tempo real da infraestrutura monitorada, por equipamento;

- Estrutura de contas com múltiplas empresas vinculadas e controle de acesso por perfil de permissão (Administrador, Gerente e Usuário comum);
- Emissão de alertas automáticos via Slack quando os recursos monitorados se aproximarem ou ultrapassarem os limites definidos;
- Abertura automática de chamado no Jira em caso de incidente crítico, com geração de log da aplicação para compor o chamado;
- Geração de um executável .jar responsável pela lógica da aplicação e pelos logs;
- Site institucional responsivo, conectado ao banco de dados;

Este documento também formaliza, nas seções 8 a 17, a solução técnica, o modelo de dados, as regras de negócio e as interfaces produzidas ao longo do desenvolvimento, que detalham e operacionalizam o escopo acima descrito.

6. Premissas e Restrições

Premissas

Para a correta operação do StreamGuard, as seguintes condições são consideradas verdadeiras:

- As ferramentas de comunicação e gestão de incidentes (Slack e Jira) estarão disponíveis e configuradas para integração durante o desenvolvimento;
- As empresas clientes fornecerão acesso ao servidor a ser monitorado para instalação do script de coleta;
- Os limites (mínimo/máximo) de cada métrica serão definidos pelo cliente responsável pelo componente monitorado, de acordo com sua própria capacidade operacional.

Restrições

O projeto opera dentro dos seguintes limites:

- Prazo fixo: o projeto deve seguir rigorosamente o cronograma de 3 sprints definido para a entrega final entre 30/nov e 07/dez;
- Ambiente de teste limitado: por não ser um ambiente de produção real, os testes de carga e os picos de utilização serão simulados, podendo não refletir com precisão o comportamento de uma infraestrutura real em larga escala;

7. Requisitos do Sistema

7.1 Requisitos Funcionais

Os requisitos funcionais definem as capacidades e comportamentos que o sistema deve oferecer aos seus usuários:

RF01. O sistema deve capturar dados de uso de CPU, RAM, disco e rede da máquina monitorada por meio de um script Python.

RF02. O sistema deve permitir o cadastro dos componentes de infraestrutura que serão monitorados, por meio de telas Web.

RF03. O sistema deve apresentar uma dashboard com gráficos e indicadores de uso de CPU, memória, disco e rede em tempo real, por equipamento monitorado.

RF04. O sistema deve emitir alertas automáticos via Slack quando os recursos monitorados atingirem limites pré-definidos.

RF05. O sistema deve abrir automaticamente um chamado no Jira quando um incidente crítico for identificado.

RF06. O sistema deve exibir o status dos principais componentes monitorados, classificado em OK, Alerta ou Crítico.

RF07. O site institucional do projeto deve estar conectado ao banco de dados e exibir informações atualizadas sobre o projeto.

RF08. O sistema deve permitir que uma conta de usuário esteja vinculada a mais de uma empresa, com seleção da empresa/equipamento a ser monitorado antes da exibição do dashboard.

RF09. O sistema deve controlar o acesso às funcionalidades de acordo com o perfil de permissão do usuário (Administrador, Gerente ou Usuário comum).

RF10. O sistema deve permitir a configuração de quais métricas são coletadas por componente, sua unidade e seu intervalo de coleta.

RF11. O sistema deve permitir o cadastro, edição e desativação de funcionários (usuários) vinculados a uma empresa, incluindo a definição do respectivo perfil de permissão.

RF12. O sistema deve manter um histórico consultável dos alertas disparados, incluindo componente, métrica, valor atingido e horário.

7.2 Requisitos Não Funcionais

Os requisitos não funcionais estabelecem os critérios de qualidade, desempenho e operação do sistema:

RNF01. O sistema de monitoramento deve funcionar de forma contínua durante o período de observação, sem interrupções não planejadas.

RNF02. O painel e as telas de cadastro devem ser intuitivos, exigindo o mínimo de treinamento para uso dos usuários.

RNF03. O site institucional do projeto deve se adaptar corretamente a diferentes tamanhos de tela.

RNF04. A arquitetura de captura e persistência de dados deve permitir a inclusão de novos componentes monitorados sem a necessidade de redesenho do sistema.

RNF05. O código-fonte e os scripts de captura devem ser organizados e documentados, permitindo fácil manutenção pela equipe.

RNF06. A captura e persistência de dados não deve gerar sobrecarga perceptível na máquina monitorada.

RNF07. A URL do Incoming Webhook do Slack e demais credenciais de integração devem ser armazenadas em variáveis de ambiente ou serviço de gerenciamento de segredos, nunca em texto plano no código.

RNF08. O tempo entre a leitura de uma métrica fora do limite e o disparo da notificação correspondente deve ser o menor possível, de modo a preservar o caráter preventivo do sistema.

8. Arquitetura da Solução

A solução foi organizada em camadas independentes e desacopladas, de modo a atender ao requisito RNF04 (extensibilidade sem redesenho) e a permitir que cada parte da arquitetura evolua isoladamente. A Figura 1 ilustra o fluxo geral de dados, da coleta na infraestrutura do cliente até a notificação da equipe responsável.

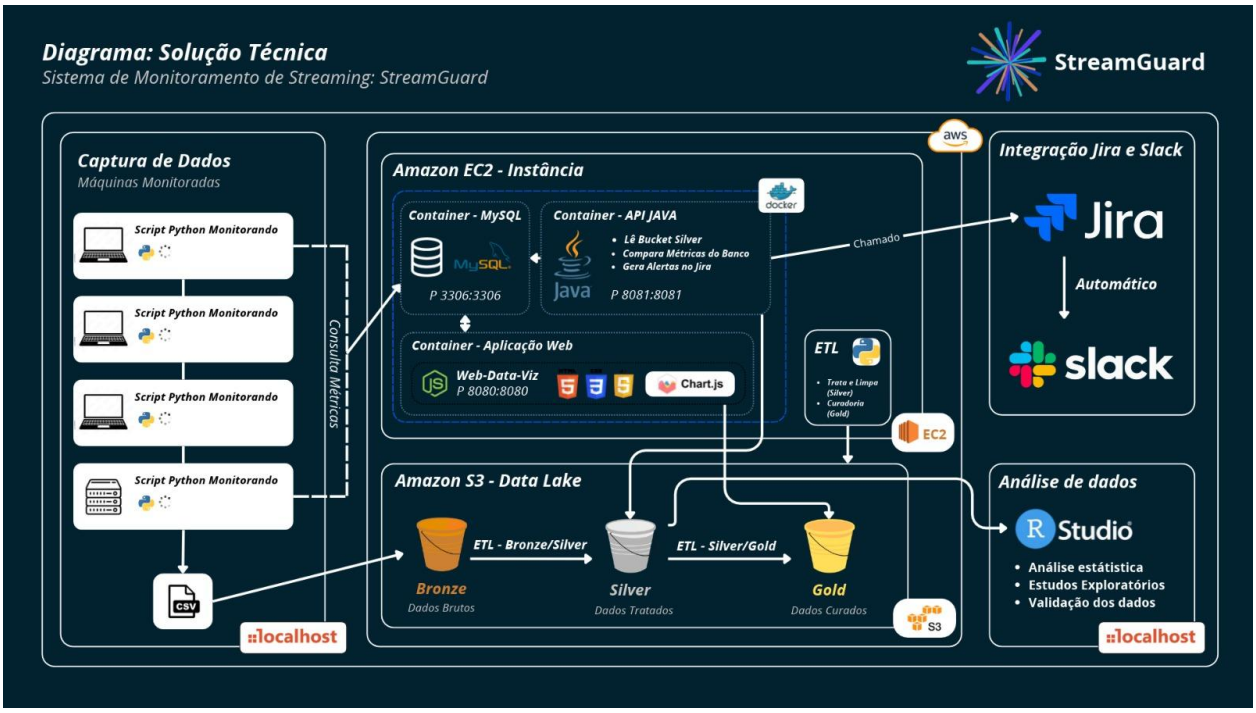


Figura 1 — Diagrama da solução técnica do StreamGuard

A Figura 1 detalha a solução técnica. As máquinas monitoradas executam o script Python de captura e gravam os dados em arquivos CSV, que são enviados ao data lake no Amazon S3. Em uma instância Amazon EC2, três contêineres Docker sustentam a aplicação: o MySQL (banco de dados), a API Java (motor de alertas, que lê o bucket, compara as métricas com os limites armazenados no banco e gera os alertas no Slack e o chamado no Jira) e a aplicação web "Web-Data-Viz" (Node.js, HTML, CSS, JavaScript e Chart.js), responsável pelas telas e gráficos. Uma rotina de ETL em Python também executa na EC2 e trata os dados armazenados no data lake, que são consumidos no RStudio para análise estatística, estudos exploratórios e validação dos dados.

8.1 Camada de coleta

Um script em Python, baseado na biblioteca psutil, é instalado no servidor do cliente e coleta periodicamente as métricas de hardware (CPU, memória, disco e rede) dos componentes cadastrados. O intervalo de coleta é configurável por métrica (tela de Configurações › Métricas), sem necessidade de alteração de código.

8.2 Camada de backend e persistência

O backend recebe as métricas coletadas, valida os dados e os persiste no banco de dados relacional. É também o backend que expõe os dados consumidos pela aplicação web.

8.3 Motor de alertas

O motor de alertas, implementado na API Java, lê os dados coletados e compara cada leitura com os limites mínimo e máximo definidos em config_metrica para aquele componente. Caso o limite seja ultrapassado, o motor classifica a severidade (Alerta ou Crítico) e aciona a camada de notificação.

8.4 Camada de notificação e rastreabilidade

Quando um limite é ultrapassado, a notificação é enviada ao Slack por meio de Incoming Webhooks. Em paralelo, incidentes críticos geram automaticamente um chamado no Jira. Um componente .jar, em execução em uma instância EC2, é responsável pela lógica de geração de logs da aplicação, que são armazenados em um bucket S3 para consulta futura e composição do chamado. A seção 11 detalha esta camada.

8.5 Camada de apresentação

A aplicação web (dashboard) consome os dados do backend e apresenta o estado da infraestrutura em tempo real, com navegação por empresa e por equipamento, além das telas de cadastro, alertas, perfil e configurações detalhadas na seção 13.

9. Modelo de Dados

O modelo lógico de dados (Figura 2) reflete diretamente as regras de negócio do sistema: uma empresa possui vários usuários e vários equipamentos; cada equipamento é formado por componentes; cada componente possui métricas monitoráveis; e cada métrica possui uma configuração de limites (config_metrica) usada pelo motor de alertas.

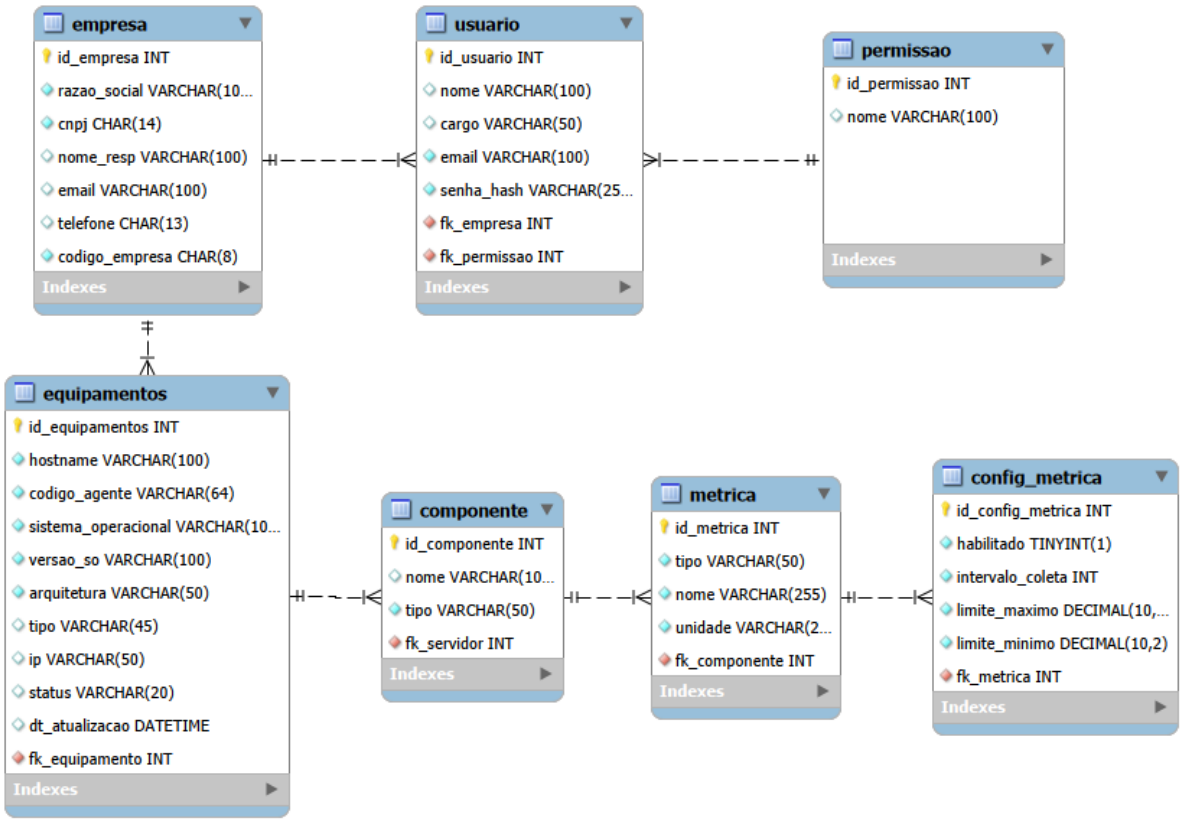


Figura 2 — Modelagem DER do StreamGuard

9.1 Entidades principais

Entidade	Finalidade	Relacionamentos
empresa	Cliente do StreamGuard; unidade à qual usuários e equipamentos pertencem.	1:N com usuario e com equipamentos

Entidade	Finalidade	Relacionamentos
usuario	Pessoa com acesso ao sistema; carrega cargo, credenciais e vínculo de permissão.	N:1 com empresa; N:1 com permissao
permissao	Perfil de acesso (Administrador, Gerente, Usuário comum).	1:N com usuario
equipamentos	Servidor/máquina física ou virtual monitorada, com dados de hostname, SO e status.	N:1 com empresa; 1:N com componente
componente	Recurso monitorado dentro de um equipamento (ex.: CPU, RAM, disco, serviço).	N:1 com equipamentos; 1:N com metrica
metrica	Tipo de dado coletado para um componente (ex.: uso %, latência, temperatura).	N:1 com componente; 1:N com config_metrica
config_metrica	Parâmetros de coleta e limites (mínimo/máximo) usados pelo motor de alertas.	N:1 com metrica

10. Regras de Negócio

10.1 Contas, empresas e permissões

RN01. Uma conta de usuário pode estar vinculada a mais de uma empresa; ao acessar o sistema, o usuário deve escolher qual empresa deseja monitorar antes de visualizar qualquer dashboard.

RN02. Cada usuário possui exatamente um perfil de permissão (Administrador, Gerente ou Usuário comum), definido na tabela permissao e vinculado no cadastro do funcionário.

RN03. Apenas o perfil Administrador — a conta principal da empresa — pode alterar Configurações (Métricas e Funcionários) e excluir equipamentos ou componentes.

RN04. O perfil Gerente pode cadastrar, visualizar e editar equipamentos e componentes, mas não pode excluí-los nem acessar a área de Configurações.

RN05. O perfil Usuário comum possui acesso somente de leitura: pode visualizar dashboards, detalhes e alertas, mas não pode cadastrar, editar ou excluir nada.

10.2 Equipamentos, componentes e métricas

RN06. Um equipamento pertence a uma única empresa; um componente pertence a um único equipamento; uma métrica pertence a um único componente.

RN07. Cada componente pode ter uma ou mais métricas monitoradas, ativadas ou desativadas individualmente na tela de Configurações › Métricas, sem necessidade de redesenho do sistema (RNF04).

RN08. Toda métrica monitorada deve possuir ao menos um limite (mínimo e/ou máximo) configurado em config_metrica; sem limite configurado, o motor de alertas não pode classificar a severidade daquela métrica.

10.3 Alertas e notificações

RN09. O estado de um componente é classificado em três severidades — OK (verde), Alerta (laranja) e Crítico (vermelho) — de acordo com a proximidade da leitura em relação aos limites configurados.

RN10. Um alerta é gerado sempre que uma leitura de métrica ultrapassa o limite configurado para a severidade Alerta ou Crítico.

RN11. Todo alerta de severidade Crítico deve, além da notificação no Slack, gerar automaticamente um chamado no Jira.

RN12. Os logs gerados pelo componente .jar durante um incidente devem ser armazenados no bucket S3 e permanecer associados ao respectivo chamado do Jira para consulta futura.

11. Integração com Ferramentas Externas (Slack e Jira)

Esta seção resume as decisões técnicas de integração com o Slack; o detalhamento completo — incluindo comparação entre modelos de integração e considerações de segurança — está no documento complementar "StreamGuard — Integração com o Slack: Fundamentos e Aplicação no Sistema", produzido pela equipe.

11.1 Por que Incoming Webhooks

Existem duas formas principais de um sistema externo publicar mensagens no Slack: Incoming Webhooks, de mão única, e a API completa do Slack (Bot Token/OAuth), bidirecional. O StreamGuard adota Incoming Webhooks pelos seguintes motivos:

- O caso de uso é estritamente de notificação de mão única — o sistema precisa avisar a equipe responsável, e ninguém precisa responder por dentro do Slack;
- A implementação é significativamente mais simples, dispensando a criação de um Slack App e o gerenciamento de tokens OAuth;

11.2 Estrutura da mensagem (payload JSON)

O corpo de uma notificação enviada via Incoming Webhook é um objeto JSON simples. Exemplo de payload correspondente a um alerta crítico de CPU:

```
{ "text": "🔴 [CRÍTICO] Componente: Transcoding Server 01" }
```

11.3 Jira — abertura automática de chamado

Quando um alerta crítico é identificado (RN11), a API Java chama a API REST do Jira Cloud para criar automaticamente um chamado, associando o componente, a métrica, o valor atingido e o log correspondente armazenado no S3. Essa automação reduz o tempo entre a detecção técnica do problema e o início do atendimento pela equipe responsável.

11.4 Limitações conhecidas

- Um Incoming Webhook está sempre vinculado a um único canal fixo, definido no momento de sua criação — não é possível escolher o canal de destino dinamicamente a cada envio;
- Não há confirmação de leitura: o sistema sabe se a mensagem foi entregue ao Slack, mas não se um responsável efetivamente a visualizou;
- A comunicação é unidirecional: o alerta não pode ser reconhecido diretamente pelo Slack.

12. Fluxo de Alertas e Notificações

A notificação via Slack não é um evento isolado — é a etapa final de uma cadeia que começa na coleta de métricas do servidor do cliente. O fluxo completo funciona da seguinte forma:

1. O script Python, instalado no servidor do cliente, coleta periodicamente métricas de hardware (CPU, memória, disco, rede) dos componentes cadastrados e grava os dados em arquivos CSV, enviados ao bucket no S3;
2. A API Java lê os dados no bucket e compara cada métrica com os limites (mínimo e máximo) configurados pelo cliente para aquele componente (config_metrica);
3. Quando um limite é ultrapassado, o sistema identifica a empresa, o equipamento e o componente correspondentes, por meio dos relacionamentos entre as tabelas envolvidas;
4. A API Java monta o payload JSON da notificação, com as informações do alerta: nome do componente, métrica, valor atingido e horário;
5. A API Java realiza a requisição HTTP POST para a URL do Incoming Webhook do Slack;
6. Em paralelo, caso o alerta seja Crítico, é aberto um chamado no Jira e gerado um log da aplicação (via componente .jar rodando na EC2), enviado a um bucket S3 para armazenamento — permitindo que a equipe consulte o histórico do incidente a partir da notificação recebida.

13. Interfaces do Sistema — Wireframes

A equipe produziu wireframes de baixa fidelidade para validar o fluxo de navegação antes da definição visual final. As telas cobrem a jornada completa do usuário: da seleção da empresa a ser monitorada até a configuração de métricas e funcionários, passando pelo controle de permissões por perfil.

- Conta StreamGuard: painel com as empresas vinculadas à conta, indicador de status geral por empresa (OK, com alerta ou crítico) e seleção de qual empresa monitorar;
- Dashboard Geral: visão de um equipamento específico, com seletor de equipamento, alertas daquele equipamento posicionados acima dos gráficos, indicadores de CPU/RAM/Disco/Rede e status dos componentes;
- Detalhes: listagem de todos os componentes cadastrados do equipamento selecionado, com filtros, status colorido e ações conforme a permissão do usuário;
- Alertas: histórico de alertas disparados, configuração de limites/thresholds por métrica e status das integrações com Slack e Jira;
- Perfil do usuário: dados pessoais, segurança (troca de senha) e preferências de notificação;
- Configurações: ponto de entrada para as telas de Métricas (quais métricas são coletadas, unidade e intervalo) e Funcionários (gestão de usuários e permissões da empresa).

O padrão de cores adotado para status segue a mesma classificação de severidade das regras de negócio (seção 10.3): verde para OK, laranja para Alerta e vermelho para Crítico, aplicado de forma consistente em indicadores, badges e itens de alerta em todas as telas.



Figura 3 — Wireframe do Dashboard Geral de um equipamento

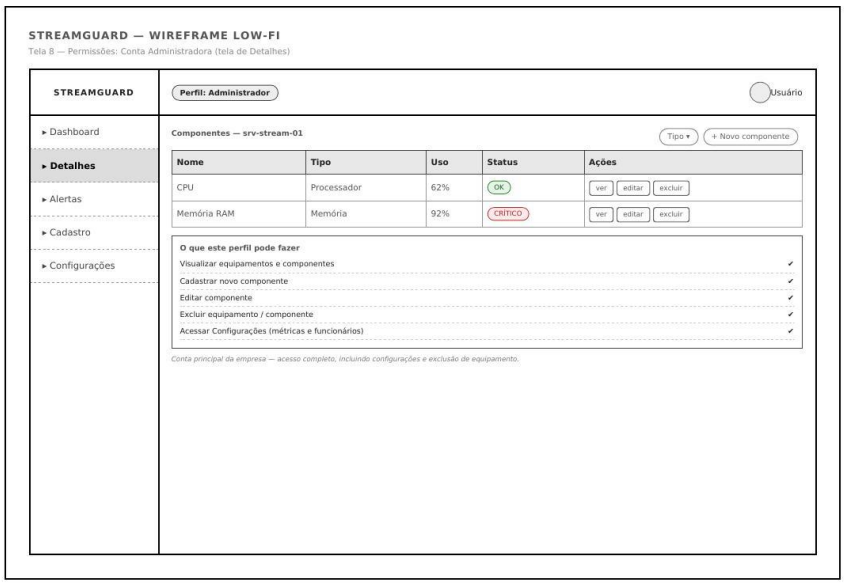


Figura 4 — Wireframe da tela de Detalhes sob o perfil Administrador

As demais telas da jornada, incluindo as variações da tela de Detalhes por perfil de permissão, são apresentadas a seguir.

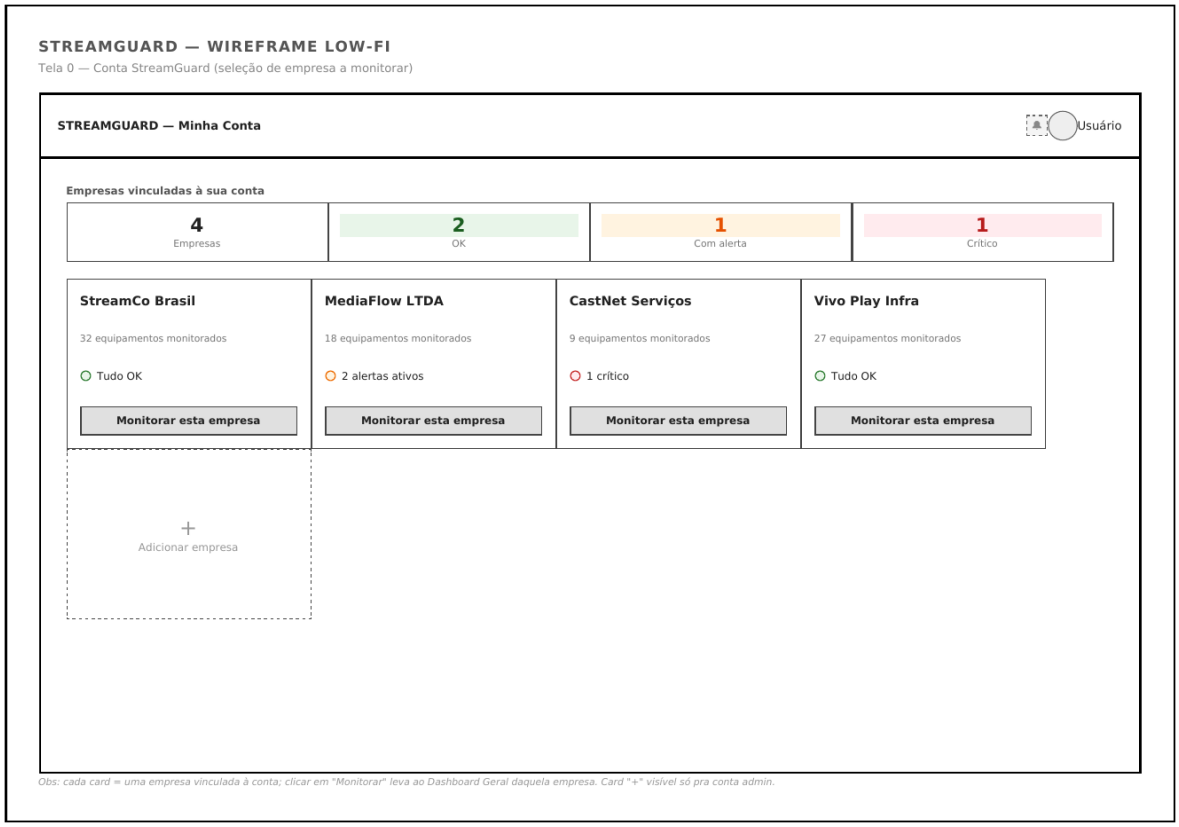


Figura 5 — Wireframe da tela Conta StreamGuard (seleção da empresa a monitorar)

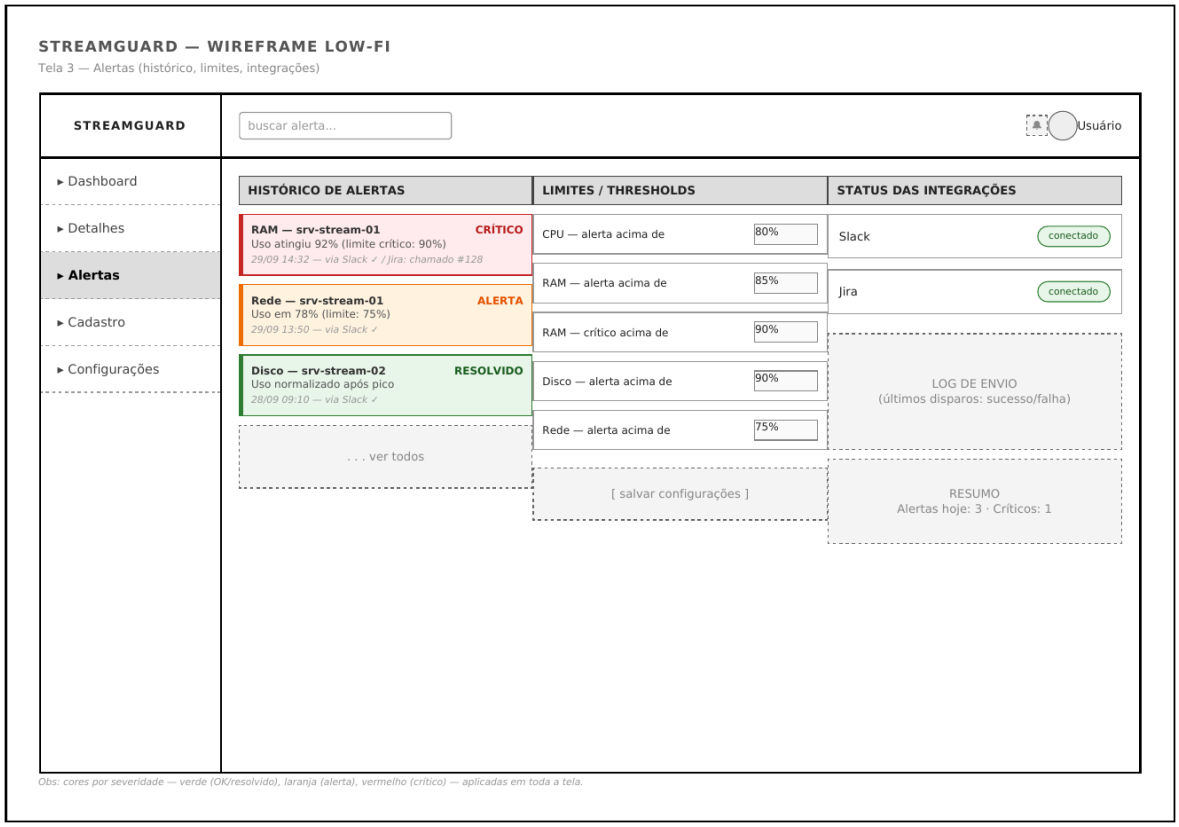


Figura 6 — Wireframe da tela de Alertas (histórico, limites e integrações)

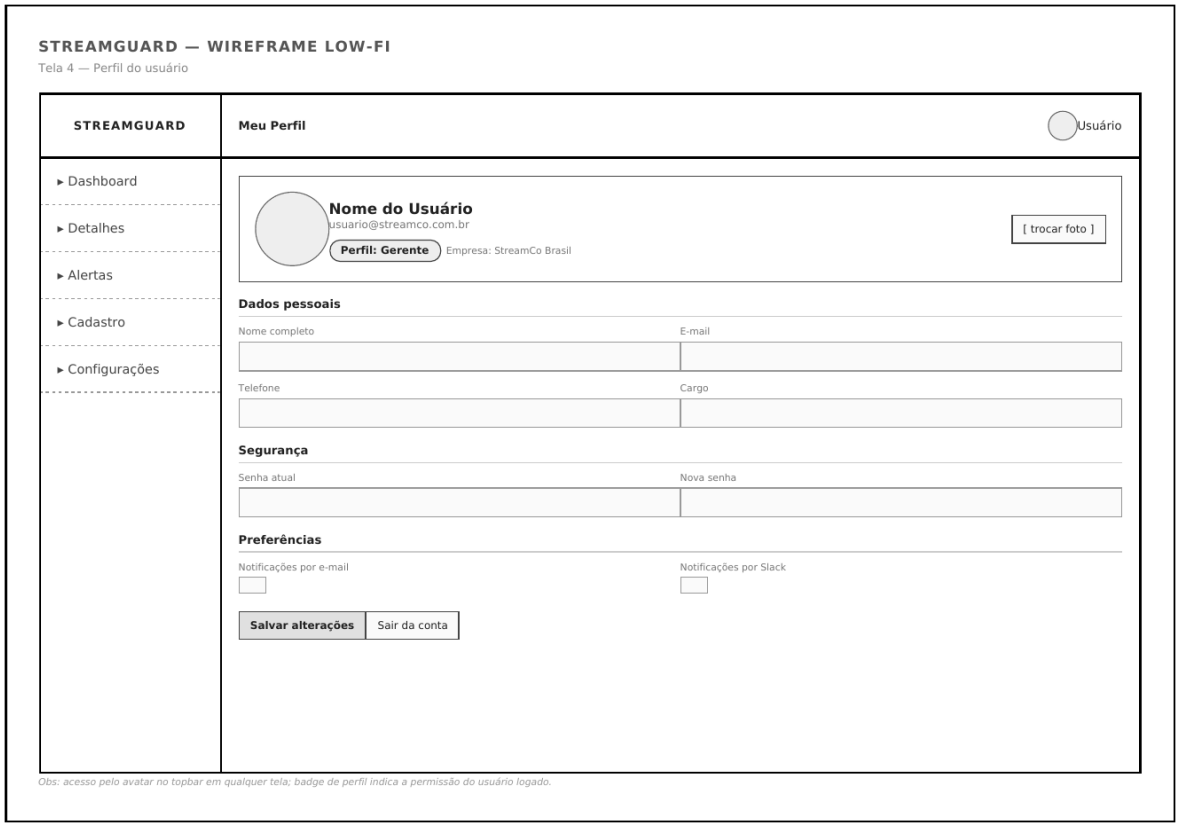


Figura 7 — Wireframe do Perfil do usuário

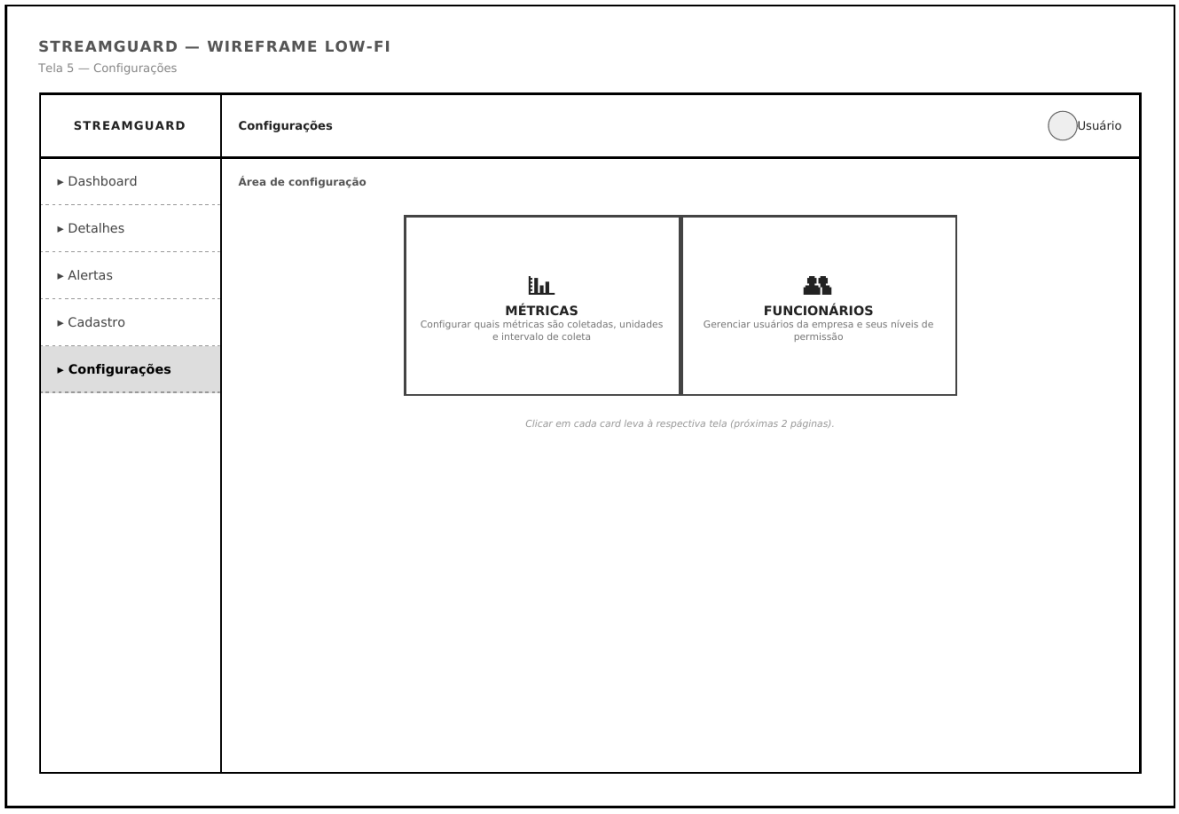


Figura 8 — Wireframe da tela de Configurações

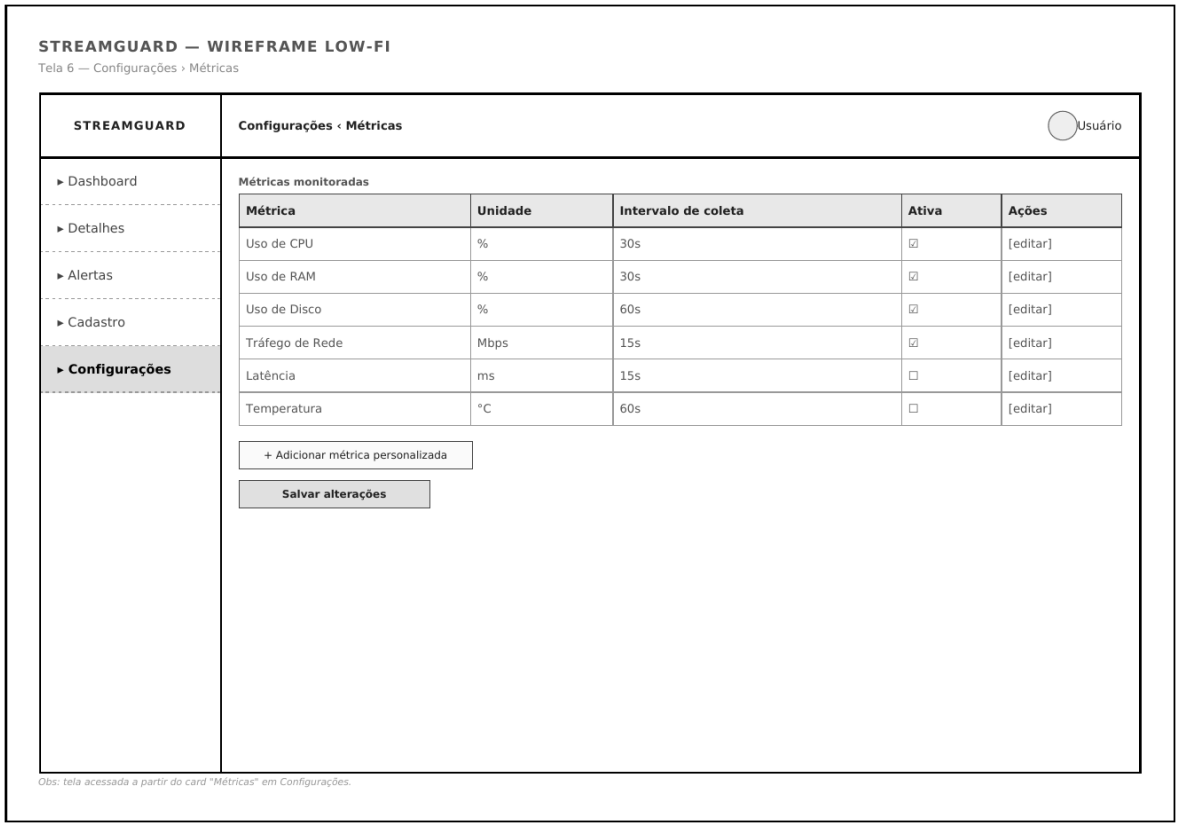


Figura 9 — Wireframe de Configurações › Métricas



Figura 10 — Wireframe de Configurações › Funcionários

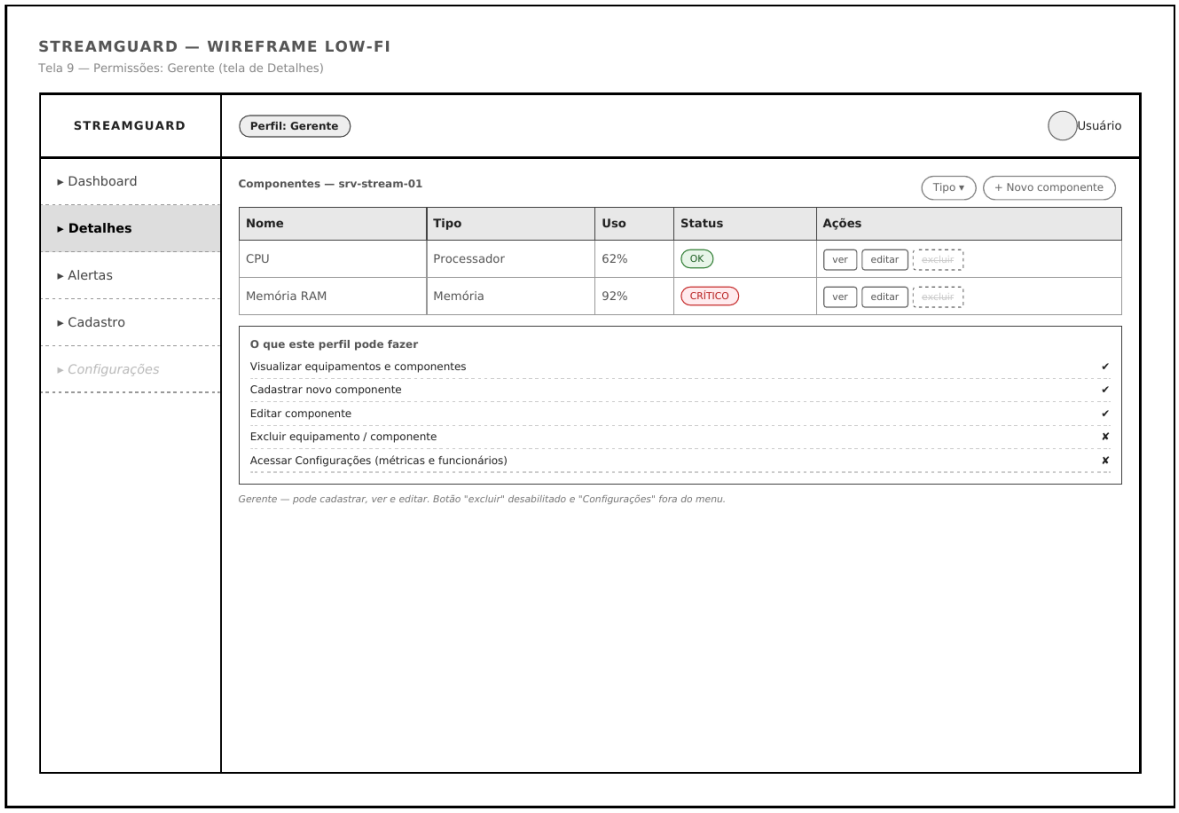


Figura 11 — Wireframe da tela de Detalhes sob o perfil Gerente

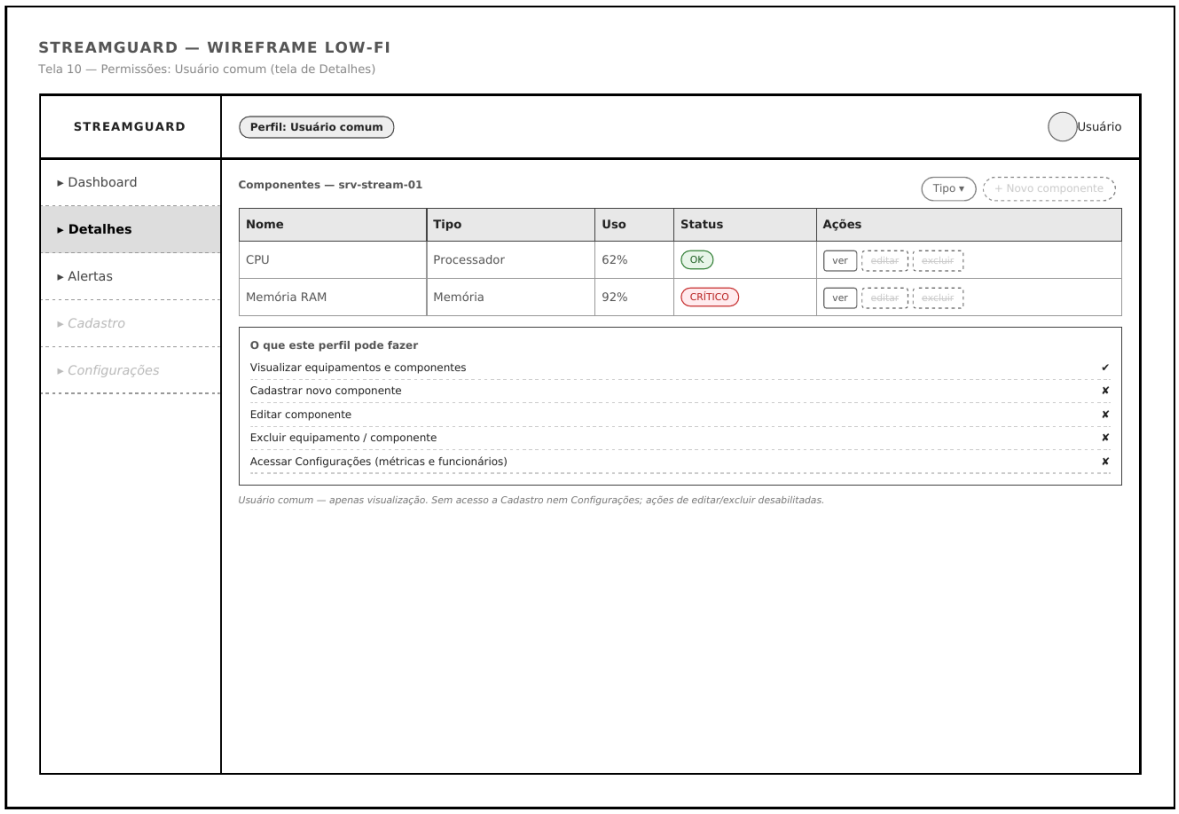


Figura 12 — Wireframe da tela de Detalhes sob o perfil Usuário comum

14. User Stories

As histórias de usuário foram levantadas a partir de duas personas centrais do sistema: o Gestor de Infraestrutura, responsável por acompanhar e configurar o monitoramento, e o Analista de Dados, responsável por analisar o comportamento histórico da infraestrutura.

14.1 Persona: Gestor de Infraestrutura (Maicon)

Persona	Maicon — Gestor de Infraestrutura
Resumo	Responsável pelo estado geral dos servidores da empresa. Precisa enxergar rapidamente o que está bom, o que está degradado e o que está crítico, e agir antes que o público seja afetado.
Objetivos	• Visualizar o estado geral dos servidores; • Cadastrar servidores e escolher componentes e métricas a monitorar; • Definir limites mínimos e máximos para as métricas; • Ser alertado e notificado antes que a situação cause indisponibilidade.
Dores	• Descobrir problemas somente depois que o público já foi afetado; • Falta de visão consolidada de toda a infraestrutura em momentos de pico; • Dificuldade para priorizar ações quando vários componentes estão sob estresse.
Como usa o StreamGuard	Dashboard do equipamento com alertas em destaque, telas de cadastro e configurações, e recebimento de alertas no Slack e de chamados no Jira.

- Eu, Maicon, como gestor de infraestrutura, quero visualizar o estado geral dos servidores da empresa, para identificar rapidamente possíveis indisponibilidades ou degradações.
- Eu, Maicon, como gestor de infraestrutura, quero cadastrar os servidores que serão monitorados, para manter a infraestrutura da empresa organizada na plataforma.
- Eu, Maicon, como gestor de infraestrutura, quero selecionar quais componentes de cada servidor serão monitorados, para adequar o monitoramento às necessidades de cada servidor.
- Eu, Maicon, como gestor de infraestrutura, quero selecionar quais métricas serão coletadas, para monitorar somente informações relevantes.
- Eu, Maicon, como gestor de infraestrutura, quero definir limites mínimos e máximos para as métricas, para que anomalias no comportamento sejam identificadas automaticamente.
- Eu, Maicon, como gestor de infraestrutura, quero ter alertas sobre leituras que ultrapassam os limites configurados, para agir antes que a situação cause uma indisponibilidade.
- Eu, Maicon, como gestor de infraestrutura, quero ser notificado sobre situações críticas, para tomar conhecimento rapidamente de possíveis impactos no meu serviço.

14.2 Persona: Analista de Dados (Kleber)

Persona	Kleber — Analista de Dados
Resumo	Analisa o comportamento histórico da infraestrutura para identificar padrões de utilização e relacionar anomalias a incidentes, apoiando a gestão de problemas.
Objetivos	• Visualizar as métricas coletadas e consultar o histórico; • Selecionar períodos de análise, como eventos de alta relevância; • Comparar o comportamento entre servidores; • Relacionar períodos acima dos limites com incidentes.
Dores	• Dados dispersos, sem tratamento e difíceis de recortar por período; • Dificuldade para reconstruir as condições de um incidente; • Falta de gráficos que evidenciem tendências e comportamentos anormais.
Como usa o StreamGuard	Telas de métricas, alertas e detalhes, consulta de histórico e análise dos dados do data lake no RStudio.

- Eu, Kleber, como analista de dados, quero visualizar as métricas coletadas de um servidor, para analisar seu comportamento e utilização de recursos.
- Eu, Kleber, como analista de dados, quero consultar o histórico das métricas para identificar padrões de utilização ao longo do tempo.
- Eu, Kleber, como analista de dados, quero selecionar diferentes períodos de análise, para estudar o comportamento da infraestrutura em intervalos específicos, como eventos de alta relevância.
- Eu, Kleber, como analista de dados, quero visualizar os dados em gráficos, para facilitar a identificação de tendências e comportamentos anormais.
- Eu, Kleber, como analista de dados, quero comparar o comportamento de servidores, para identificar diferenças de utilização entre as máquinas.
- Eu, Kleber, como analista de dados, quero identificar períodos em que as métricas ultrapassaram seus limites, para relacionar anomalias com possíveis incidentes.
- Eu, Kleber, como analista de dados, quero consultar as métricas correspondentes ao período de um incidente, para auxiliar na análise das condições que contribuíram para ele.

15. Lean UX

O Lean UX organiza o projeto a partir do problema de negócio, dos usuários e clientes envolvidos, das ideias de solução e dos benefícios esperados para o negócio e para o usuário. Ele originou as personas e as histórias de usuário apresentadas na seção 14.

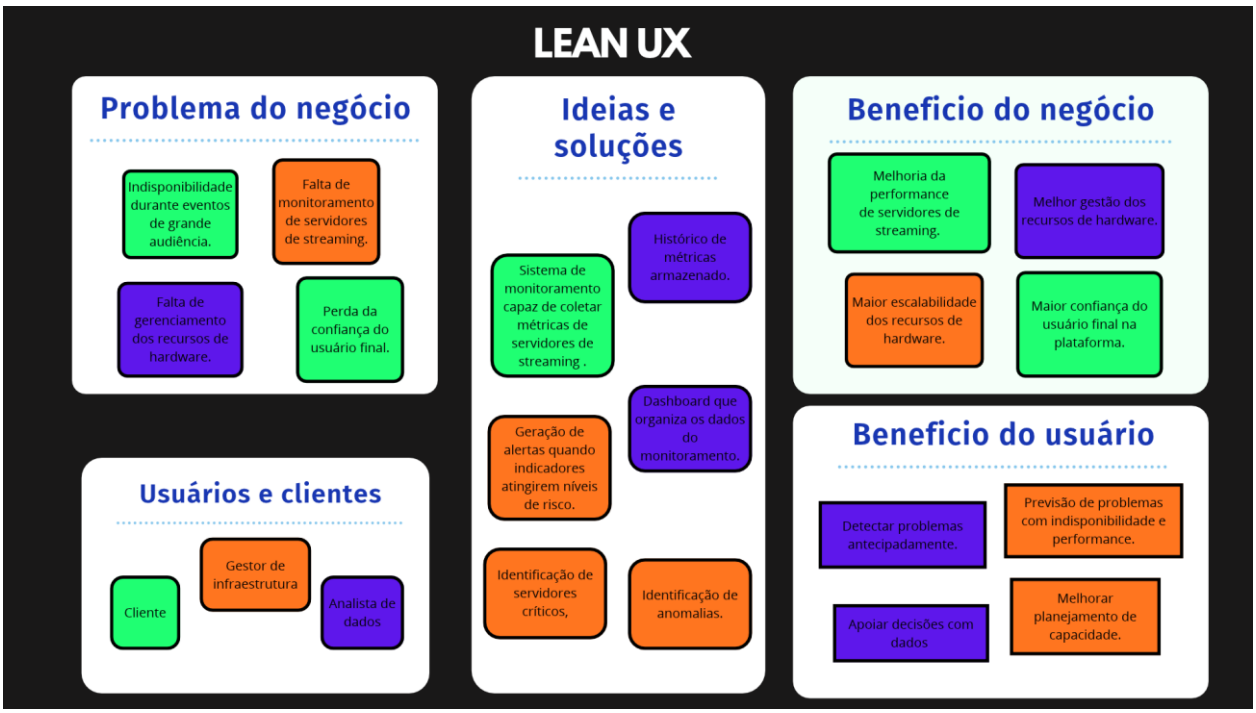


Figura 13 — Lean UX do StreamGuard

Quadro	Conteúdo
Problema do negócio	Indisponibilidade durante eventos de grande audiência; falta de monitoramento de servidores de streaming; falta de gerenciamento dos recursos de hardware; perda da confiança do usuário final.
Usuários e clientes	Cliente, gestor de infraestrutura e analista de dados.
Ideias e soluções	Sistema de monitoramento capaz de coletar métricas de servidores de streaming; histórico de métricas armazenado; dashboard que organiza os dados do monitoramento; geração de alertas quando indicadores atingirem níveis de risco; identificação de servidores críticos; identificação de anomalias.
Benefício do negócio	Melhoria da performance de servidores de streaming; melhor gestão dos recursos de hardware; maior escalabilidade dos recursos de hardware; maior confiança do usuário final na plataforma.
Benefício do usuário	Detectar problemas antecipadamente; previsão de problemas com indisponibilidade e performance; apoiar decisões com dados; melhorar o planejamento de capacidade.

16. Diagrama de Caso de Uso

O diagrama de caso de uso apresenta as interações dos três atores do sistema com as funcionalidades do StreamGuard: o Gestor de Infraestrutura, o Analista de Dados e o próprio Sistema, que recebe os dados do servidor, os históricos e os usuários.

DIAGRAMA DE CASO DE USO

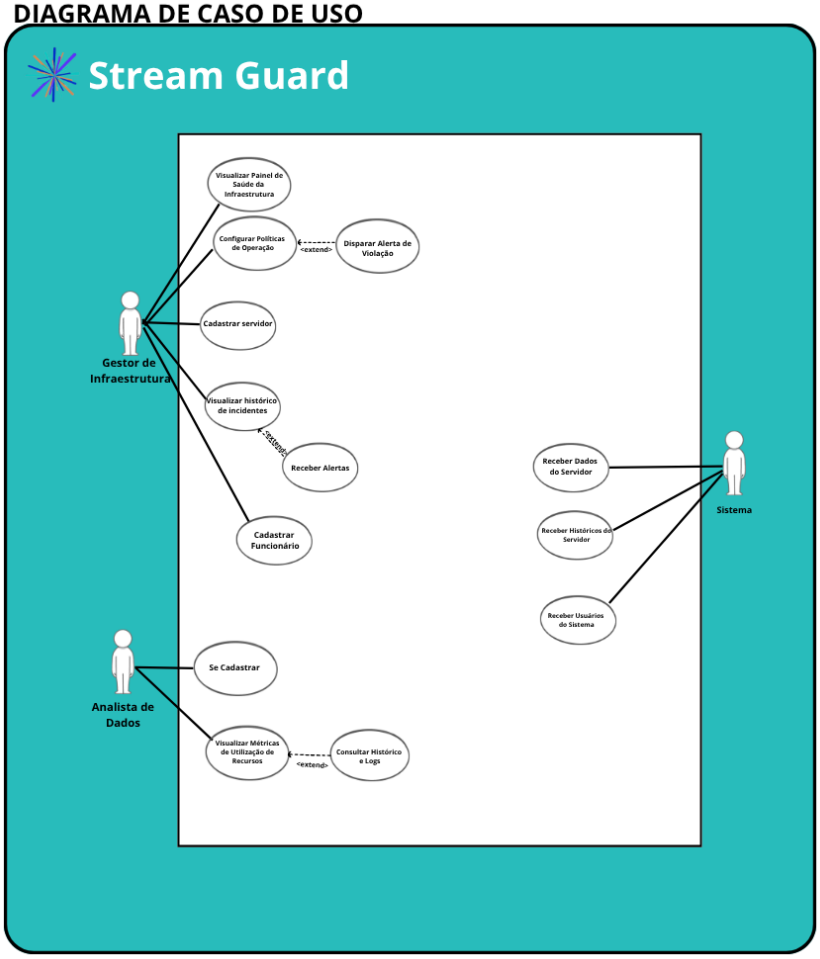


Figura 14 — Diagrama de caso de uso do StreamGuard

Ator	Casos de uso	Relacionamentos
Gestor de Infraestrutura	• Visualizar Painel de Saúde da Infraestrutura • Configurar Políticas de Operação • Cadastrar servidor • Visualizar histórico de incidentes • Cadastrar Funcionário	• «extend»: Disparar Alerta de Violação estende Configurar Políticas de Operação • «extend»: Receber Alertas estende Visualizar histórico de incidentes
Analista de Dados	• Se Cadastrar • Visualizar Métricas de Utilização de Recursos	«extend»: Consultar Histórico e Logs estende Visualizar Métricas de Utilização de Recursos
Sistema	• Receber Dados do Servidor • Receber Históricos do Servidor • Receber Usuários do Sistema	Ator responsável por receber e persistir os dados enviados pelo script de captura e pelo cadastro de usuários.

17. BPMN — Monitoramento Preventivo de Infraestrutura

O diagrama BPMN descreve o processo de monitoramento preventivo do StreamGuard, distribuído em três raias: o Gestor de Infraestrutura, o Sistema StreamGuard e o Analista de Dados. O processo parte do cadastro e da ativação do monitoramento, passa pela coleta contínua de métricas e pela decisão sobre o limite crítico e termina com a ação de infraestrutura tomada pelo gestor.

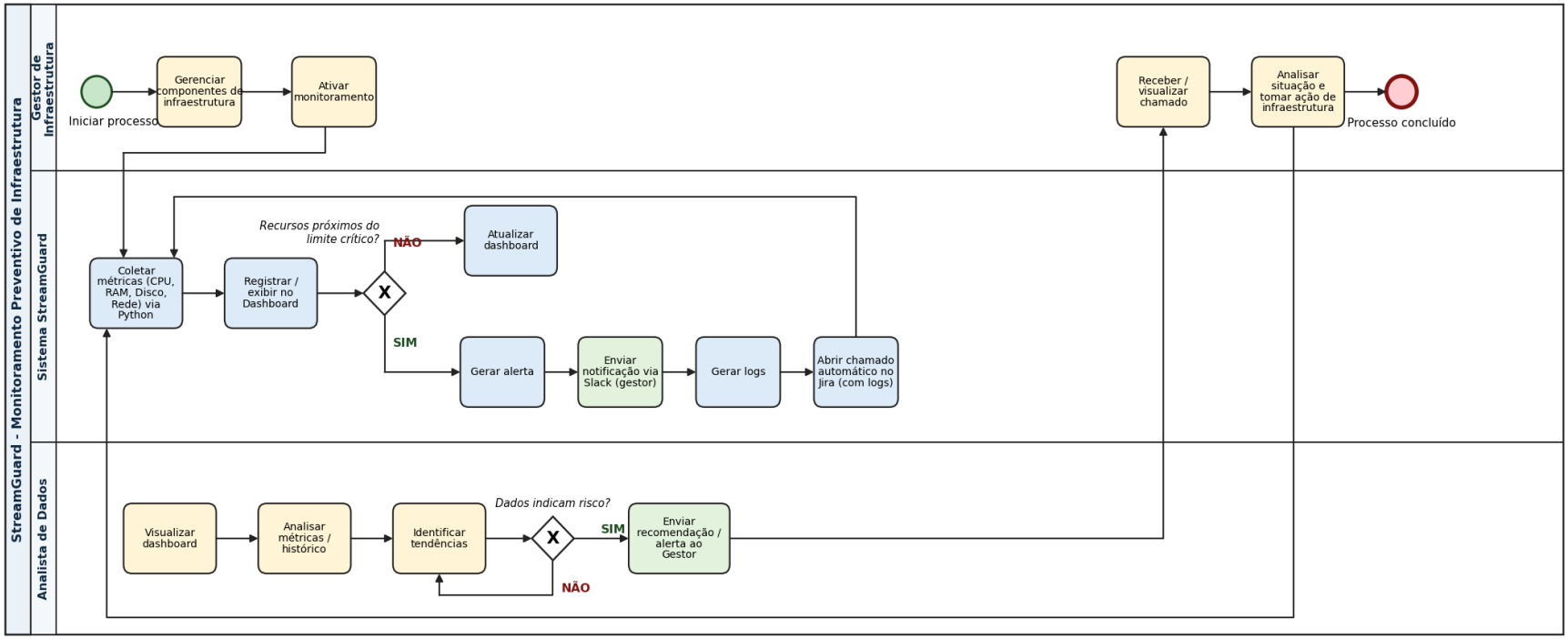


Figura 15 — BPMN do processo de monitoramento preventivo de infraestrutura

Raia	Atividades
Gestor de Infraestrutura	Gerenciar componentes de infraestrutura → Ativar monitoramento. Ao final do processo: Receber / visualizar chamado → Analisar situação e tomar ação de infraestrutura → Processo concluído.

Raia	Atividades
Sistema StreamGuard	Coletar métricas (CPU, RAM, disco e rede) via Python → Registrar / exibir no dashboard → decisão “Recursos próximos do limite crítico?”. **Não:** Atualizar dashboard. **Sim:** Gerar alerta → Enviar notificação via Slack (gestor) → Gerar logs → Abrir chamado automático no Jira (com logs), retornando à coleta de métricas.
Analista de Dados	Visualizar dashboard → Analisar métricas / histórico → Identificar tendências → decisão “Dados indicam risco?”. **Não:** retorna a Identificar tendências. **Sim:** Enviar recomendação / alerta ao Gestor.

18. Metodologia

A equipe adotou a Metodologia Ágil como framework central de gestão e desenvolvimento. Essa escolha se justifica pela natureza iterativa do projeto — com entregas parciais e funcionais a cada sprint — e pela necessidade de adaptação rápida a mudanças de requisitos ou imprevistos durante o desenvolvimento.

A metodologia está estruturada em três pilares:

Comunicação

As cerimônias de Daily ocorrem todos os dias úteis. Nesses encontros, os membros da equipe compartilham o progresso de cada tarefa, identificam e removem impedimentos e alinham prioridades para os próximos dias. A regularidade dessas reuniões garante que todos os integrantes estejam cientes do estado atual do projeto e das dependências entre as atividades.

Transparência e Visibilidade

O Planner é utilizado como ferramenta principal de gestão visual do backlog. Cada tarefa é registrada com descrição, categoria (Essencial, Importante ou Desejável) e dificuldade utilizando a sequência de Fibonacci. Essa estrutura permite à equipe priorizar corretamente e identificar gargalos com agilidade.

Melhoria Contínua

Retrospectivas ao final de cada sprint permitem que a equipe reflita sobre o que funcionou bem, o que pode melhorar e quais ajustes de processo ou comunicação devem ser realizados. Esse ciclo de aprendizado contínuo é fundamental para aumentar a produtividade e a qualidade das entregas ao longo do projeto.

19. Diagrama de Requisitos — Backlog

O backlog do projeto consiste em uma lista organizada contendo todas as funcionalidades, requisitos, melhorias e tarefas necessárias para o desenvolvimento do sistema StreamGuard. Sua principal finalidade é auxiliar no planejamento das atividades da equipe, permitindo a priorização das funcionalidades mais importantes para cada etapa do projeto. Para o gerenciamento do backlog, foram utilizadas ferramentas como planilhas no Excel e a plataforma Planner.

O tamanho de cada tarefa segue a sequência de Fibonacci: PP = 3, P = 5, M = 8, G = 13 e GG = 21 pontos. A classificação indica a importância da tarefa (Essencial ou Importante) e a prioridade define a ordem de execução dentro da sprint. As disciplinas são identificadas por sigla: PI (Projeto de Inovação), AS (Análise de Sistemas), LP (Linguagem de Programação), Prog (Programação), SO (Sistemas Operacionais) e Calc. Comp. (Cálculo Computacional). A Sprint 1 partiu de 185 pontos e a Sprint 2 de 236 pontos, conforme os gráficos de burndown da seção 20; todas as tarefas listadas foram concluídas.

19.1 Backlog da Sprint 1 (SPT01)

Nº	Disc.	Requisito	Classif.	Tam.	Pts	Prio.	Resp.	Status
1	PI	Contexto + Justificaiva do projeto	Essencial	PP	3	1	Marina	Feito

Nº	Disc.	Requisito	Classif.	Tam.	Pts	Prio.	Resp.	Status
2	PI	Documentação Inicial do projeto	Essencial	P	5	2	Marina	Feito
3	PI	BackLog Excel + Gráfico de Burndown	Importante	P	5	1	Caio	Feito
4	PI	BackLog no Planner	Importante	P	5	3	Leonardo	Feito
5	PI	Entendimento do Slack + Integração	Essencial	P	5	1	Kaue	Feito
6	A.S	Proto + Persona	Importante	P	5	2	Marina	Feito
7	PI	Pesquisar / propor inovação do projeto	Importante	P	5	2	Leonardo	Feito
8	A.S	Lean UX Canvas	Importante	P	5	3	Anderson	Feito
9	PI	Lista de Requisitos Backlog	Essencial	M	8	1	Kaue	Feito
10	A.S	História do Usuário	Essencial	M	8	1	Marina	Feito
11	Prog	Page Home - Site Institucional	Importante	M	8	1	Caio	Feito
12	Prog	Page About + Contato - Site institucional	Importante	M	8	2	Leonardo	Feito
13	Prog	Page Login + Cadastro - Site institucional	Essencial	M	8	1	Kaue	Feito
14	A.S	Lista de Dados Necessários/DER	Importante	M	8	2	Kaue/Leo/Anderson	Feito
16	SO	Instância provisionada em EC2	Essencial	P	5	3	Leonardo	Feito
17	A.S	Política de gestão de acessos	Essencial	M	8	1	Todos	Feito
18	SO	Python configurada na EC2	Importante	P	5	2	Anderson	Feito
19	SO	Jar na EC2	Essencial	P	5	2	Kaue	Feito
20	Calc. Comp.	Capturar Dados via Python	Essencial	M	8	1	Todos	Feito
21	Calc.Comp	Capturar os processos virtuais e discretizar	Essencial	G	13	3	Todos	Feito
22	Calc. Comp.	Armazenar localmente a captura de dados em Arquivo .CSV	Essencial	P	5	1	Todos	Feito
23	Prog	API - (Web-data-viz) integrado com o front	Importante	GG	21 + 21	1	Todos	Feito

19.2 Backlog da Sprint 2 (SPT02)

Nº	Disc.	Requisito	Classif.	Tam.	Pts	Prio.	Resp.	Status
1	PI	Conexão do backend com o MySQL	Essencial	P	5	1	Caio	Feito
2	AS	Prototipação das telas do MVP	Essencial	G	13	2	Anderson	Feito
3	PI	Tela Config Metricas + empresa	Essencial	G	13	1	Caio	Feito
4	PI	BackLog no Planner	Importante	G	13	3	Leonardo	Feito
5	PI	Tela de alerta	Essencial	G	13	1	Marina	Feito
6	PI	Diagrama de solução tecnica	Essencial	M	8	2	Caio	Feito

Nº	Disc.	Requisito	Classif.	Tam.	Pts	Prio.	Resp.	Status
7	LP	Configuração da aws	Essencial	M	8	2	Caio	Feito
8	LP	Leitura do bucket	Essencial	G	13	3	Kaue	Feito
9	PI	Implementação do jira + criacao automatica de issue	Essencial	G	13	1	Leo	Feito
10	PI	Integração com slack	Essencial	M	8	1	Leo	Feito
11	AS	BPMN - as-is to-be	Importante	M	8	1	Leo	Feito
12	AS	Caso de uso	Essencial	M	8	2	Marina	Feito
13	AS	DER Vers2	Essencial	M	8	1	Marina	Feito
14	PI	Tela de componentes	Importante	G	13	2	Kaue	Feito
15	Calc. Comp.	Captura de dados	Importante	M	8	1	Anderson	Feito
16	Calc. Comp.	Mineração de dados	Essencial	GG	21	3	Anderson	Feito
17	Calc. Comp.	Analise exploratoria descritiva	Essencial	M	8	1	Kaue	Feito
18	Calc. Comp.	Analise exploratoria temporal	Importante	M	8	2	Kaue	Feito
19	Calc. Comp.	Gráficos e ranking de criticidade	Essencial	M	8	2	Anderson	Feito
20	PI	Tela Gerenciamento StreamGuard	Essencial	G	13	2	Caio	Feito
21	PI	Tela Perfil Usuario	Essencial	G	13	2	Anderson	Feito
22	PI	Tela DashGeral	Essencial	G	13	1	Leonardo	Feito

20. Gráfico de Burndown

O burndown foi desenvolvido para melhor controle do andamento do projeto. Os pontos entregues, baseados na sequência de Fibonacci, são registrados em uma tabela e, para melhor visualização, refletidos em um gráfico de burndown, permitindo um melhor entendimento de como está a evolução do sistema a cada dia.

Os gráficos abaixo foram gerados a partir dos dados da planilha de backlog: a linha tracejada representa o ritmo ideal de entrega e a linha contínua, os pontos que ainda restavam pendentes em cada dia.

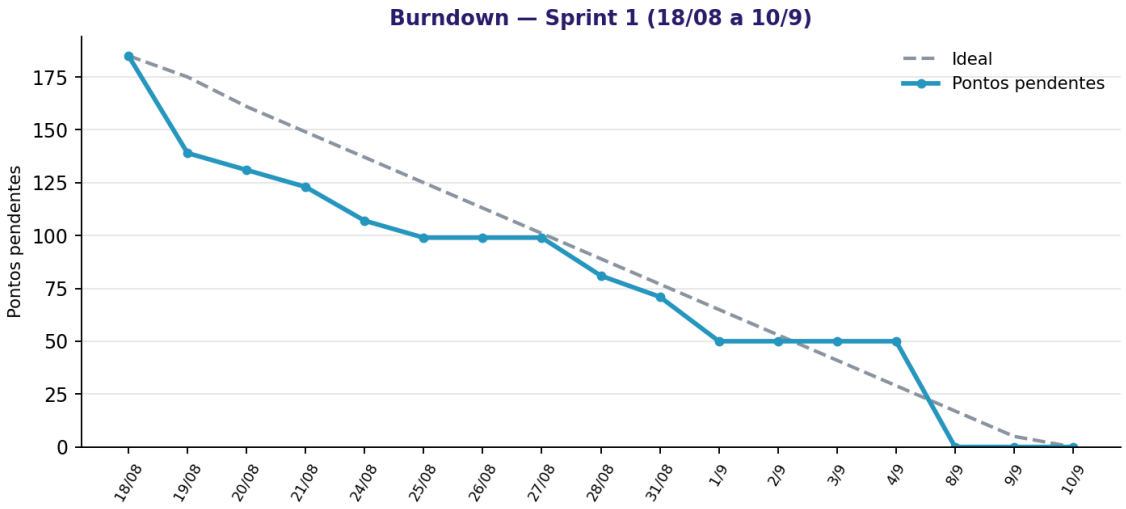


Figura 16 — Burndown da Sprint 1 (18/08/2026 a 10/09/2026)

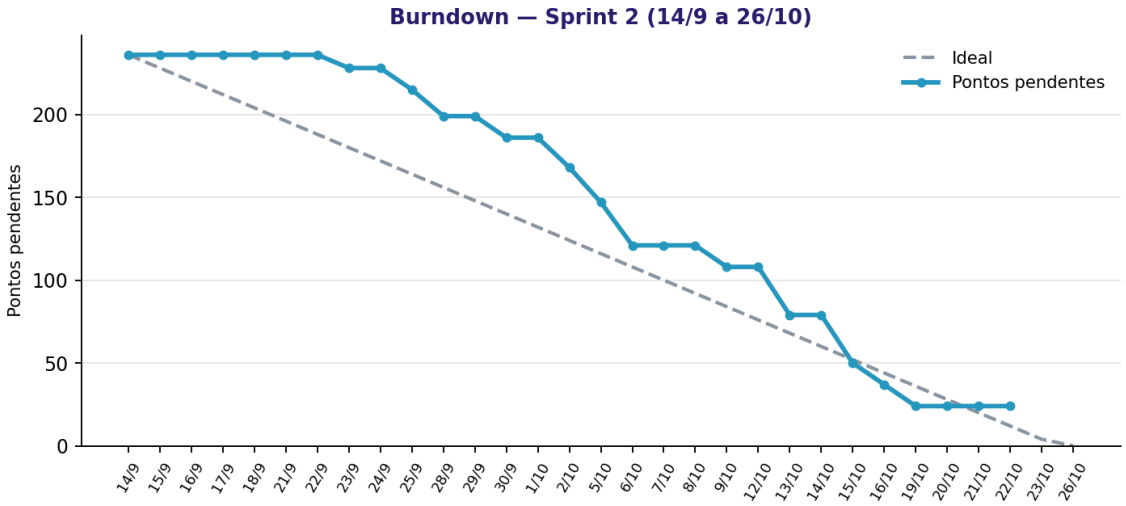


Figura 17 — Burndown da Sprint 2 (14/09/2026 a 26/10/2026)

21. Considerações Finais

O StreamGuard nasce da constatação de que picos de audiência em grandes eventos ao vivo, como observado durante a Copa do Mundo de 2026, expõem fragilidades de infraestrutura que impactam diretamente a experiência do espectador, a receita publicitária e a confiança do público na plataforma. Ao propor um painel automatizado de monitoramento, capaz de antecipar sinais de sobrecarga antes que afetem a transmissão, o projeto busca transformar uma atuação hoje reativa em uma atuação preventiva por parte das equipes técnicas.

Com a formalização da arquitetura, do modelo de dados, das regras de negócio e das interfaces apresentadas neste documento (seções 8 a 17), o projeto consolida a ponte entre a concepção inicial e a implementação técnica, reduzindo ambiguidades para as próximas sprints.

Para as próximas sprints, a equipe seguirá o cronograma definido, com foco no desenvolvimento da aplicação, na preparação da infraestrutura e, posteriormente, nos ajustes finais e no deploy em produção, mantendo como fio condutor o compromisso, a participação ativa e a melhoria contínua estabelecidos como boas práticas do projeto.

22. Referências Bibliográficas

Contexto e justificativa

- Discussão 1 — InternetBrasil (Reddit) — https://www.reddit.com/r/InternetBrasil/comments/1uhlfpb/caz%C3%A9tv_travando_youtube_vivofibra_horario_de_pico/
- Discussão 2 — InternetBrasil (Reddit) — https://www.reddit.com/r/InternetBrasil/comments/1uiy6t9/youtube_n%C3%A3o_aguenta_a_caz%C3%A9tv_em_jogos_do_brasil/
- Google Cloud — Live Streaming com Media CDN e Load Balancer — https://cloud.google.com/blog/products/networking/live-streaming-with-media-cdn-and-google-cloud-load-balancer/
- YouTube — Como o Brasil fez história no YouTube durante a Copa do Mundo FIFA 2026 — https://blog.youtube/intl/pt-br/news-and-events/como-o-brasil-fez-historia-no-youtube-durante-a-copa-do-mundo-da-fifa-2026/
- AppsFlyer — Global Soccer Marketing Report — https://www.appsflyer.com/pt/resources/reports/global-soccer-marketing-report/
- Tutor — Impacto dos problemas de buffering nos serviços de streaming — https://www.tutor.do/noticia/47-dos-consumidores-cancelaram-a-assinatura-de-um-servico-de-streaming-devido-a-problemas-de-buffering
- Tecflow — Impacto financeiro do downtime — https://tecflow.com.br/2026/06/16/downtime-custa-600-bilhoes-ataques-ciberneticos/

Fundamentação técnica e normativa

- AXELOS — ITIL Foundation: ITIL 4 Edition (Monitoramento de Serviços, Gestão de Incidentes e Gestão de Problemas) — https://www.axelos.com/certifications/itil-service-management
- ISO/IEC 20000-1:2018 — Information technology — Service management — Part 1: Service management system requirements — https://www.iso.org/standard/70636.html
- Slack — Sending messages using Incoming Webhooks — https://docs.slack.dev/messaging/sending-messages-using-incoming-webhooks/
- Atlassian — Jira Cloud REST API (criação de chamados) — https://developer.atlassian.com/cloud/jira/platform/rest/v2/api-group-issues/#api-rest-api-2-issue-post
- psutil — Cross-platform library for process and system monitoring (documentação oficial) — https://psutil.readthedocs.io/en/latest/
- AWS — Amazon S3 User Guide — https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html
- AWS — Amazon EC2 Documentation — https://docs.aws.amazon.com/ec2/